VEIL NETWORK

Privacy-Preserving Ethereum Layer 2 Infrastructure

TECHNICAL RESEARCH WHITEPAPER

Draft v0.1 | 2 November 2026

Privacy without compromise — a research direction, not a deployed product.

Project status

VEIL is an independent early-stage research concept. No production network has been deployed; no token has been issued; no public token sale has taken place; and no independent security audit has been completed. The design in this document is proposed and subject to technical, legal, economic, and security review.

This document is intended to support technical discussion and ecosystem feedback. It is not an investment offer, token-sale document, financial promotion, legal opinion, or commitment to launch.

Document Control

Field	Description
Project	VEIL Network
Document	Privacy-Preserving Ethereum Layer 2 Infrastructure — Technical Research Whitepaper
Version	Draft v0.1
Date	2 November 2026
Stage	Concept and research
Intended audience	Ethereum researchers, ZK engineers, L2 developers, ecosystem grant reviewers, and potential technical collaborators
Status	Non-binding research proposal; all architecture and token assumptions are provisional

Contents

1. Executive Summary
2. Problem Statement and Design Goals
3. Scope, Assumptions, and Non-Goals
4. Proposed System Architecture
5. Privacy State Model: Notes, Commitments, and Nullifiers
6. Private Transaction Lifecycle
7. Public and Private Asset Balances
8. USDC Deposit and Withdrawal Architecture
9. Sequencer and Prover Networks
10. Fee Model and Native VEIL Token Considerations
11. Governance and Upgrade Model
12. Security Model and Threat Analysis
13. Developer and User Experience
14. Research Roadmap and Deliverables
15. Evaluation and Acceptance Criteria
16. Risks, Dependencies, and Open Questions
17. Provisional Token Design (Non-Binding)
18. Grant-Suitable Open-Source Work Packages

Appendix A. Glossary

Appendix B. Reference Starting Points

Reading note: This paper distinguishes proposed design choices from unresolved research questions. No feature should be interpreted as implemented unless explicitly stated as such.

1. Executive Summary

VEIL Network is a proposed privacy-focused Layer 2 (L2) system built on Ethereum. Its research objective is to explore how zero-knowledge (ZK) proofs, private asset accounting, and programmable execution can be combined while retaining Ethereum as the settlement and security anchor.

The central product hypothesis is that users and applications need a choice between transparent and confidential interactions. A user may want a public balance for ordinary DeFi activity, a private balance for confidential transfers, or the ability to move value between the two states under verifiable rules. VEIL proposes to study a dual-state asset model in which public balances are represented transparently and private balances are represented by cryptographic notes.

The proposed system includes an L2 execution environment, a transaction-ordering component, a distributed proving workflow, an Ethereum settlement and bridge layer, wallet and SDK components, and indexing services designed to avoid unnecessary disclosure of private note contents.

The initial research will prioritize a narrow, open-source prototype rather than a complete production network. The first objective is to demonstrate a private transfer lifecycle, including note selection, witness generation, proof generation, nullifier checks, commitment insertion, and recipient note discovery. Bridge flows and native-token fee mechanisms will be investigated as subsequent workstreams.

VEIL is not presented as a new cryptographic primitive. It intends to evaluate and integrate existing, publicly documented approaches and open-source components, including relevant Ethereum rollup infrastructure and ZK developer tooling. The project must clearly distinguish upstream components from original VEIL engineering and must comply with the relevant licenses.

The proposed native VEIL token is a future design consideration only. It has not been issued, distributed, or offered for sale. The core research prototype should not depend on a token sale and should be useful as an open-source contribution independently of any future token model.

2. Problem Statement and Design Goals

2.1 Problem statement

Public blockchains provide transparent state and publicly observable transaction histories. This transparency supports independent verification, but it can expose balances, transaction amounts, counterparties, financial relationships, and behavioral patterns. Even when an application hides some fields, timing, transaction graph structure, wallet reuse, and public interactions can reveal information.

Privacy systems introduce a different set of trade-offs. Users need strong correctness guarantees, reliable asset accounting, understandable recovery and viewing-key models, and practical interoperability with existing assets and applications. A privacy layer must also preserve clear settlement and withdrawal rules and must avoid making unverifiable claims about anonymity.

2.2 Design goals

- Confidentiality by default for explicitly private transaction fields, while preserving public verifiability of state transitions.
- User-selectable transparency: support both public and private asset representations and controlled conversion between them.
- Ethereum settlement: use Ethereum as the intended settlement layer, subject to the selected rollup architecture and its security assumptions.
- Sound asset accounting: prevent double spending, unauthorized minting, and unbacked withdrawals.
- Practical usability: enable wallet note discovery, recipient recovery, fee estimation, and clear transaction status.

- Open-source research: publish specifications, reference implementations, test vectors, and reproducible test results.
- Incremental decentralization: document the trust assumptions of sequencing, proving, data availability, and upgrades at every stage.

2.3 Design principles

VEIL should minimize trusted roles, avoid custom cryptography where audited primitives exist, make trust assumptions explicit, and prefer simple state machines over discretionary administrator controls. Privacy must be measured as a system property rather than inferred from the presence of a ZK proof. The project should publish known limitations and should not market a prototype as production-grade privacy.

3. Scope, Assumptions, and Non-Goals

3.1 Initial scope

- A research prototype for private value transfer on an Ethereum-anchored L2.
- A dual public/private asset-accounting model.
- A conceptual USDC bridge with explicit deposit, claim, withdrawal, and accounting states.
- A transaction lifecycle using notes, commitments, nullifiers, and ZK proofs.
- A documented sequencing and proving workflow.
- An investigation of fee abstraction and the feasibility of a native VEIL-denominated fee experience.

3.2 Assumptions requiring validation

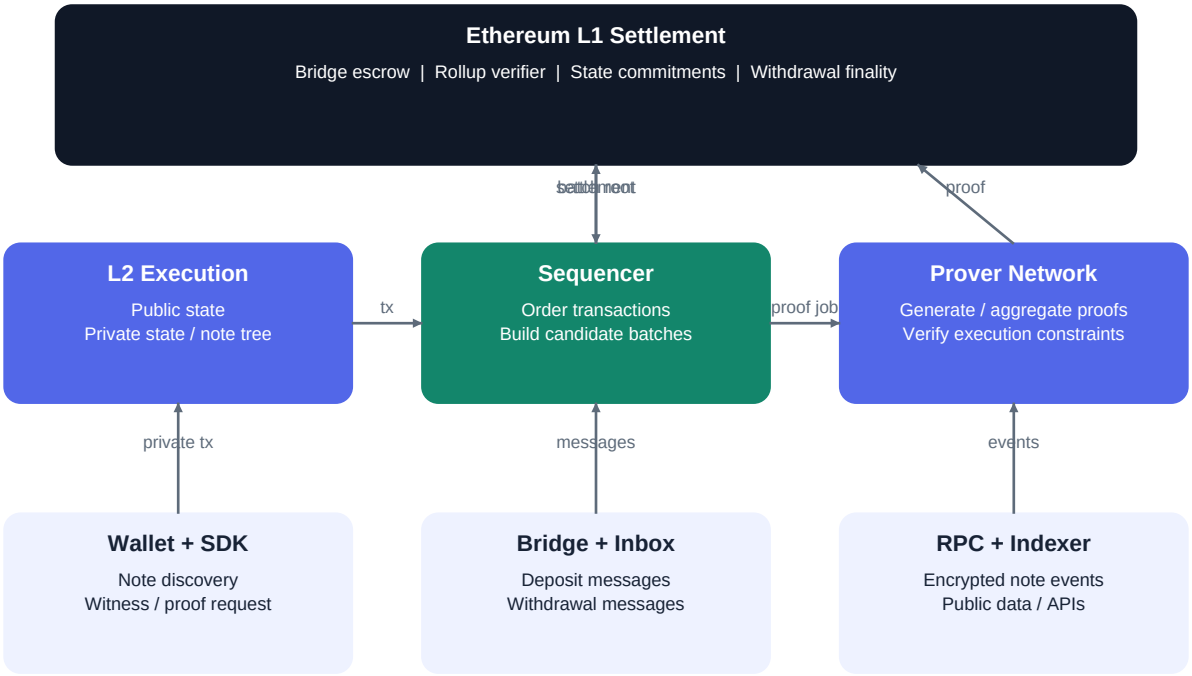
Assumption	Status / validation required
Ethereum settlement	Target architecture; exact rollup framework and verifier design remain to be selected.
Data availability	Must select and document calldata, blobs, or another supported DA arrangement; no DA guarantee is claimed here.
ZK proving system	Must evaluate circuit language, proof system, recursion/aggregation, verifier cost, and developer maturity.
Aztec and Noir	Research references, not a claim of compatibility, endorsement, or a commitment to fork a specific release.
USDC representation	Must select canonical bridge, escrow-backed representation, or an issuer-supported interoperability route.
Native VEIL fees	Not implemented; compatibility with upstream fee accounting must be proven before claiming protocol-native gas.
Privacy guarantees	Depend on circuit correctness, note encryption, wallet behavior, network metadata, anonymity set, and operational security.

3.3 Non-goals

- Creating a new ZK proving system or cryptographic primitive.
- Claiming complete anonymity or protection against all network-level metadata analysis.
- Launching a public token sale as part of the initial research scope.
- Promising a specific transaction throughput, fee level, token price, or investment return.
- Claiming audits, partnerships, regulatory approvals, or deployed contracts that do not exist.

4. Proposed System Architecture

The architecture is conceptual. The exact execution engine, proof system, settlement contracts, and data-availability design must be selected after a technical feasibility review.



Conceptual architecture only — component boundaries and sequencing remain subject to research.

4.1 Ethereum settlement layer

The Ethereum layer is intended to anchor rollup state commitments, verify validity proofs or other protocol-required commitments, manage bridge escrow and withdrawal finality, and provide an auditable record of system-level state transitions. The precise settlement design depends on the chosen rollup framework. The document does not claim that a verifier contract or bridge has been deployed.

4.2 L2 execution and state

The L2 execution environment processes public and private state transitions. Public state can use conventional account or contract storage. Private state is represented by commitments to notes, with spent-note prevention through nullifiers. The state model must define how public calls interact with private circuits and how cross-domain messages are authenticated.

4.3 Sequencer

The sequencer receives transactions, applies admission and fee rules, orders transactions, and proposes candidate batches. In an initial devnet, sequencing may be centralized for operational simplicity, but the trust limitation must be visible. A later design should evaluate multiple sequencers, proposer rotation, censorship monitoring, and a forced-inclusion or escape mechanism where supported by the selected rollup stack.

4.4 Prover network

Provers execute proof-generation jobs for transactions or batches. The protocol must specify job assignment, proof format, verification, deadlines, retries, failure handling, and compensation. Prover decentralization is not achieved merely by allowing multiple operators to run software; the system also needs verifiable job accounting and a robust method for preventing duplicate or invalid reward claims.

4.5 Wallet, RPC, and indexing

Wallet software selects spendable notes, constructs witnesses, manages encryption and viewing keys, estimates fees, and submits transactions. RPC and indexing services expose public state and encrypted note events. Indexers should not need plaintext note amounts or recipient identities to perform their public indexing role. Wallets may use private scanning or delegated scanning, but any delegation introduces additional metadata and trust considerations.

5. Privacy State Model: Notes, Commitments, and Nullifiers

5.1 Note model

A private asset unit is represented by a note. A note conceptually contains an asset identifier, amount, owner authorization data, a unique randomness value, and optional encrypted metadata. The exact encoding, domain separation, and cryptographic hash function must be selected and specified with the chosen circuit system.

5.2 Commitment

A commitment binds the note contents without publishing them. A commitment is inserted into a note commitment tree or equivalent authenticated data structure. Observers can verify that a commitment exists without learning the underlying amount or owner. The circuit must prove that the commitment is correctly formed and that the note is authorized by the relevant secret.

5.3 Nullifier

When a note is spent, the wallet derives a nullifier from secret note-spending material. The nullifier is published so that the system can reject a second spend of the same note, while the construction is designed not to reveal the note contents or directly identify the originating commitment. The precise relation between note secret, nullifier key, and commitment must be circuit-constrained and reviewed for linkability risks.

5.4 Encrypted note discovery

A recipient needs enough encrypted information to detect and reconstruct notes addressed to them. The system must define note encryption, recipient keys, incoming viewing keys, outgoing viewing keys, key rotation, backup, and recovery. A viewing key may reveal transaction details to a third party; documentation must clearly state what each key can disclose and whether it can authorize spending (it should not).

5.5 Privacy boundaries

- Public chain observers may see transaction timing, batch inclusion, public fees, and public bridge interactions.
- Wallet network traffic may reveal IP addresses or behavioral patterns unless mitigated by the wallet and network stack.
- Small anonymity sets or unusual amounts can reduce practical privacy even if the proof is sound.
- Deposits and withdrawals create observable boundaries between public and private domains.
- Viewing-key sharing and wallet backups can intentionally disclose information.
- Application-level events, errors, and logs must not accidentally expose plaintext note contents.

A ZK proof establishes that a transition satisfies specified constraints; it does not by itself conceal every piece of metadata or guarantee anonymity.

6. Private Transaction Lifecycle

The following lifecycle is a proposed reference flow for a private transfer. Each step requires a concrete specification and implementation tests.

1. **Note discovery:** the sender's wallet scans encrypted note data or uses a selected scanning service to identify notes that may belong to the wallet.
2. **Spend selection:** the wallet selects notes whose total value covers the transfer amount and fee, subject to asset and authorization rules.
3. **Witness construction:** the wallet constructs private inputs, Merkle inclusion paths, ownership secrets, recipient data, change outputs, and fee inputs.
4. **Nullifier derivation:** the wallet derives nullifiers for notes to be consumed. The circuit proves that each nullifier corresponds to a valid owned note.
5. **Output commitments:** the wallet creates commitments for recipient and change notes and encrypts the note payloads for intended recipients.
6. **Proof generation:** the wallet or a proving service generates a proof that inputs exist, are authorized, are not previously spent, and balance correctly.
7. **Submission and ordering:** the transaction is submitted to the L2 interface and ordered by the sequencer under the documented admission and fee rules.
8. **Execution and state update:** the system verifies the transaction constraints, marks nullifiers as spent, and inserts new commitments.
9. **Batch proof and settlement:** the proving workflow produces the required proof or batch artifact; the rollup posts the relevant commitment and data to Ethereum according to the selected design.
10. **Recipient discovery:** the recipient wallet detects and decrypts the encrypted note, then updates its spendable-note view after the required confirmation status.

Failure states must be explicit: invalid proof, stale note path, already-spent nullifier, insufficient fee, sequencer rejection, batch reorganization, unavailable prover, and L1 settlement delay. The wallet should distinguish submitted, included, proven, settled, and safely spendable states rather than displaying a single ambiguous 'success' status.

7. Public and Private Asset Balances

VEIL proposes a dual representation for supported assets. A public balance is visible in ordinary L2 account state. A private balance is represented by one or more notes. The two representations must share a single conservation model so that value cannot be created or spent twice.

Operation	Public state effect	Private state effect	Required invariant
Public → Private	Debit public balance	Create private note commitment	Public debit equals private credit, net of disclosed fee.
Private → Private	No public amount disclosure beyond protocol metadata	Spend input notes; create recipient/change notes	Input value equals outputs plus fee.
Private → Public	Credit public balance	Spend private note	Private debit equals public credit, net of fee.
Public → Public	Ordinary transparent transfer	No private note transition	Conventional token/accounting rules apply.

Asset identifiers must be domain-separated and unambiguous across L1, L2, and token representations. Decimal handling, fee denomination, token upgrades, and unsupported token behavior must be specified. The prototype should initially support a narrowly defined test asset set rather than arbitrary token contracts.

A key design choice is whether public and private balances are two modes of one L2 asset contract or separate representations linked by a conversion protocol. This decision affects composability, accounting complexity, audit scope, and user experience and should be evaluated before implementation.

8. USDC Deposit and Withdrawal Architecture

USDC support is a proposed integration, not an existing partnership or issuer endorsement. The system must distinguish an escrow-backed L2 representation from an issuer-supported cross-chain transfer. VEIL cannot mint native USDC unless the issuer explicitly grants the required authority.

8.1 Deposit to public balance

1. The user approves the L1 bridge portal to transfer the supported USDC contract.
2. The portal transfers USDC into a designated escrow or uses the selected canonical bridge mechanism.
3. The L1 portal emits a deposit message containing the sender, recipient, asset identifier, amount, destination mode, and a unique message identifier.
4. The L2 inbox authenticates the message under the rollup's cross-domain messaging rules and rejects replayed message identifiers.
5. The user claims the deposit to a public L2 balance, or follows a separately defined private-claim path.

8.2 Deposit to private balance

A private deposit claim should create a note commitment only after the L2 system has authenticated the corresponding L1 deposit message. The claim process must bind the asset, amount, recipient authorization, and message identifier. If the recipient claims privately, the public chain may still reveal the L1 deposit amount and timing; the private claim only reduces subsequent L2 balance and transfer visibility.

8.3 Private withdrawal to Ethereum

1. The wallet selects and spends private notes, proving ownership and sufficient value.
2. The circuit constrains the withdrawal amount, destination, asset, and applicable fee.
3. The L2 system emits a withdrawal message and records the associated nullifier or withdrawal identifier.
4. The rollup reaches the required settlement/finality condition on Ethereum.
5. The L1 bridge verifies the authenticated withdrawal message and releases the escrowed USDC to the specified recipient.
6. The bridge marks the withdrawal identifier as completed to prevent duplicate release.

8.4 Accounting invariants

- Total L2 circulating representation must never exceed the amount backing it, subject to explicitly documented bridge fees and pending-message states.
- Every deposit message can be claimed at most once.
- Every withdrawal message can release funds at most once.
- Pending, finalized, cancelled, and refunded states must be mutually consistent.
- Bridge upgrades and emergency pauses must not silently confiscate user claims.

- Reconciliation tooling must compare escrow balances, pending deposits, L2 supply, pending withdrawals, and completed releases.

8.5 Bridge design alternatives

Option	Advantages	Risks / dependencies
Escrow-backed L2 representation	Clear reserve accounting; can be prototyped with test assets.	Bridge contract and message security are critical; L1/L2 representation must be reconciled.
Canonical rollup bridge	May inherit established message and finality mechanisms from the chosen rollup stack.	Tightly coupled to the selected framework; must understand its upgrade and proof assumptions.
Issuer-supported interoperability	Could preserve issuer-recognized USDC movement where supported.	Requires issuer/partner integration and compliance review; cannot be assumed available.

9. Sequencer and Prover Networks

9.1 Sequencer responsibilities

- Validate transaction envelope, signatures, fee payment, and basic admission rules.
- Order transactions and build candidate L2 blocks or batches.
- Publish required transaction data or commitments through the selected DA path.
- Provide inclusion status and predictable error reporting.
- Support monitoring for censorship, downtime, and abnormal ordering.

The initial devnet may use a single sequencer, but the whitepaper and UI must state this trust assumption. A later design should evaluate sequencer rotation, multiple operators, proposer bonds, forced inclusion, and censorship-resistance mechanisms. The selected upstream rollup architecture may constrain which mechanisms are feasible.

9.2 Prover responsibilities

- Receive proof jobs with a deterministic job identifier and public input commitment.
- Generate transaction or batch proofs using the specified proving software.
- Return proof artifacts and verifiable job metadata before a defined deadline.
- Allow independent verification of proof correctness.
- Support retry and fallback if a prover is unavailable or returns an invalid artifact.

9.3 Rewards and penalties

A future prover reward model could compensate completed, valid, timely work. Rewards should be linked to verifiable proof acceptance and should not pay twice for the same job. Slashing should be introduced only when misconduct is objectively provable and the appeal or dispute process is defined. Hardware availability, proof complexity, job size, and market competition can affect reward economics.

The research prototype should first measure proving time, memory use, hardware requirements, proof verification cost, and failure rates across representative circuits. It should not publish performance claims until results are reproducible and independently reviewable.

10. Fee Model and Native VEIL Token Considerations

10.1 Fee components

A user fee may need to cover execution, proof generation, data availability, sequencer operations, and the amortized cost of L1 settlement. The actual fee model depends on the rollup framework and the amount of data published to Ethereum.

A conceptual fee decomposition is:

> **Total L2 fee = execution component + proof component + data-availability component + amortized L1 settlement component + congestion adjustment.**

The formula is a design framework, not a deployed pricing algorithm. Each component must be measurable, and the protocol must prevent fee underpayment from creating an unbounded subsidy or insolvency risk.

10.2 VEIL-denominated gas

The intended user experience is to pay L2 fees in VEIL. However, a token transfer function alone does not make a token the protocol's native gas asset. The chosen L2 stack may have its own fee accounting, fee-credit, or resource-metering mechanism. If the team builds on an upstream system such as Aztec, the exact version's fee model and compatibility constraints must be reviewed before making a native-gas claim.

A staged approach is recommended:

1. **Prototype stage:** use the upstream framework's supported fee mechanism and test a VEIL-denominated payment adapter or paymaster only if the framework permits it safely.
2. **Integration stage:** define conversion, price-oracle, volatility, fee-reserve, and failure behavior. The user-facing VEIL amount must map to the actual resource cost without relying on an unsafe or manipulable price source.
3. **Protocol stage:** pursue true protocol-level native fee support only if the selected stack supports it or a carefully scoped protocol modification is justified, reviewed, and audited.

10.3 L1 ETH costs

Ethereum settlement transactions require ETH at the L1 layer unless the submitting infrastructure uses a separate relayer or fee sponsorship arrangement. A VEIL-denominated L2 fee therefore does not eliminate the need to fund L1 settlement. The protocol must maintain a transparent reserve and account for who bears L1 cost volatility.

10.4 Fee distribution

A future fee policy may distribute net fees among sequencer operations, prover compensation, protocol reserves, and other disclosed purposes. Any buyback or burn mechanism must be based on realized net revenue after operating costs, reserves, refunds, and obligations. It must not be marketed as a promise to support token price or guarantee returns.

11. Governance and Upgrade Model

The initial prototype should use explicit, limited administrative controls and should document who can change each parameter. Critical upgrades should have a delay and public notice period. Emergency pause authority should be narrow, logged, and subject to a published recovery process.

A proposed maturity path:

Stage	Control model	Required transparency
Research/devnet	Small, disclosed operator set; test-only assets.	Public config, deployment addresses, release notes.
Public testnet	Multisig for critical operations; timelock for non-emergency changes.	On-chain proposals, signer policy, pause reasons, upgrade notice.
Mainnet candidate	Independent security review; constrained upgrade authority.	Audit reports, upgrade policy, incident process, monitoring.
Mature network	Progressive decentralization where technically and legally appropriate.	Published governance scope, quorum rules, delegation and accountability.

Token-based governance is not assumed to be safe or appropriate at launch. Governance design should consider voter concentration, delegation, low participation, bribery, capture, emergency response, and conflicts between token holders and users. A token vote should not override mandatory security controls or legal obligations.

12. Security Model and Threat Analysis

Security work must cover the full system: circuits, execution, bridge, settlement contracts, wallets, proving infrastructure, RPC/indexers, upgrade controls, and operational processes.

Threat	Potential impact	Mitigation / research task
Circuit constraint bug	Invalid value creation, unauthorized spend, or privacy failure.	Formal specification, independent circuit review, adversarial test vectors, audit.
Nullifier or commitment flaw	Double-spend or linkability.	Domain separation, collision analysis, invariant tests, circuit review.
Bridge replay or message forgery	Duplicate release or unbacked L2 credit.	Authenticated messages, unique IDs, replay protection, reserve invariants.
Sequencer censorship or outage	Delayed inclusion and degraded liveness.	Monitoring, fallback/forced inclusion research, multi-operator design.
Prover failure or collusion	Batch delays or denial of service.	Independent verification, retries, redundancy, transparent job accounting.
Data withholding	Users may be unable to reconstruct state or exit.	DA design review, data publication guarantees, escape/recovery design.
Metadata leakage	Users' activity may be inferred despite ZK proofs.	Threat modeling, network privacy guidance, encrypted events, UX warnings.
Admin key compromise	Unauthorized upgrade, pause, or treasury action.	Multisig, timelock, key separation, monitoring, recovery procedures.
Oracle manipulation	Incorrect VEIL fee conversion or economic loss.	Avoid oracle dependency where possible; otherwise robust sources and bounds.

12.1 Audit plan

1. Publish a threat model and state-transition specification before implementation.
2. Use established libraries and minimize custom cryptographic code.
3. Run unit, integration, fuzz, and invariant tests in continuous integration.
4. Review circuit constraints separately from Solidity and bridge contracts.

5 Commission independent reviews before any production deployment.

6 Publish audit scope, findings, remediation status, and residual risks.

7 Conduct a testnet incident exercise covering pause, recovery, and withdrawal behavior.

No audit has been completed. This paper does not claim that VEIL contracts, circuits, bridge, or network are secure or production-ready.

13. Developer and User Experience

A privacy L2 must be usable by wallets and developers without requiring every application to implement cryptographic details independently. The proposed developer surface includes:

- Wallet SDK: note scanning, note selection, witness construction, proof requests, viewing-key management, fee estimation, and transaction status.
- Developer SDK: public/private asset APIs, contract interaction helpers, event decoding, and test utilities.
- RPC: transaction submission, public state queries, proof status, and authenticated cross-domain message status.
- Indexer: public block data and encrypted note payload discovery without plaintext note disclosure.
- Explorer: public transaction metadata, batch status, L1 settlement references, and privacy-preserving status views.

The explorer should not expose plaintext private amounts or recipient identities. It should clearly distinguish data that is public, encrypted, user-decryptable, or unavailable. Developer documentation must include privacy pitfalls such as address reuse, public/private conversion leakage, and accidental logging.

14. Research Roadmap and Deliverables

The schedule below is an indicative planning range, not a delivery commitment. Dates depend on team capacity, upstream dependencies, and technical findings.

Phase	Indicative window	Workstreams	Deliverables
0. Specification	0–2 months	Architecture selection, threat model, asset state model, upstream review.	Architecture decision records; protocol specification; open questions register.
1. Private-transfer prototype	2–4 months	Note model, commitment tree, nullifier logic, proof circuit, wallet test harness.	Local prototype; test vectors; reproducible demo; initial security notes.
2. Devnet integration	4–7 months	L2 execution integration, sequencer workflow, prover job flow, RPC/indexer.	Private transfer devnet; batch proof flow; integration tests.
3. Bridge and dual-balance testnet	7–10 months	Test-asset deposit/claim/withdraw, public/private conversion, accounting reconciliation.	Sepolia-connected test environment; bridge state machine; invariant test report.
4. Security and public testnet	10–14 months	Independent reviews, fuzzing, monitoring, wallet UX, recovery drills.	Public testnet candidate; audit/review reports; documented limitations.
5. Mainnet readiness research	14+ months	Governance, decentralization, economics, legal review, operational readiness.	Readiness assessment; no mainnet launch until all gates are satisfied.

Milestones should be re-estimated after Phase 0. A grant proposal should request funding for a narrow, independently useful work package rather than imply that a full production L2 can be completed within a short grant period.

15. Evaluation and Acceptance Criteria

The prototype should be considered successful only when its behavior is reproducible and independently testable. Suggested acceptance criteria include:

- A documented private transfer state machine with explicit valid and invalid transitions.
- Test vectors showing note creation, ownership proof, nullifier derivation, commitment insertion, and recipient note recovery.
- Tests demonstrating that a spent note cannot be spent again.
- Tests demonstrating conservation of value across inputs, outputs, change, and fees.
- Negative tests for invalid proofs, malformed messages, replayed deposits, and duplicate withdrawals.
- Integration tests covering deposit-to-public, deposit-to-private, private-to-private, and private-to-public flows using test assets.
- Fuzz and invariant tests for supply backing, nullifier uniqueness, withdrawal uniqueness, and vesting/fee accounting if those modules are included.
- Reproducible build and deployment instructions, dependency pinning, and documented upstream licenses.
- A public threat model and a list of known limitations.
- Independent technical feedback or review before describing the prototype as suitable for broader use.

No throughput, cost, or privacy benchmark is asserted in this draft. Any future performance results should include hardware, circuit size, proof system, data availability assumptions, sample size, and reproducible scripts.

16. Risks, Dependencies, and Open Questions

16.1 Principal risks

- Scope risk: building a complete privacy L2 is substantially larger than building a private-transfer demonstration.
- Upstream dependency risk: the selected rollup stack may not support the desired fee token, private state model, or bridge behavior without material changes.
- Cryptographic risk: circuit bugs or incorrect assumptions can undermine correctness and privacy.
- Bridge risk: cross-domain messaging and escrow accounting create critical attack surfaces.
- Privacy risk: transaction metadata, wallet behavior, and small anonymity sets can reduce practical privacy.
- Operational risk: centralized sequencing or proving can create censorship and liveness dependencies.
- Regulatory risk: token issuance, public sale, privacy features, and cross-border promotion may trigger legal obligations in multiple jurisdictions.
- Funding risk: grants and investment are uncertain; the project should be scoped so that useful open-source outputs can be delivered incrementally.

16.2 Open technical questions

1. Which existing rollup framework best supports the required privacy state model and Ethereum settlement assumptions?
2. Should VEIL build on an existing privacy L2 stack, integrate selected components, or develop a separate execution layer?
3. What data availability model provides the required recovery and exit properties?

4. Which proof system and circuit language provide the best balance of security, performance, tooling, and maintainability?
5. How should note encryption and private scanning work across wallets and devices?
6. Which bridge model can support test USDC first and a production asset later without implying issuer endorsement?
7. Can users pay fees in VEIL through a safe adapter, or would true native fee support require protocol changes?
8. What decentralization and forced-inclusion guarantees are feasible in each development phase?

16.3 Open governance and legal questions

The project has not selected a legal entity, token issuer, jurisdiction, or public distribution structure. No legal conclusion is made in this paper. Any future token issuance, sale, staking program, or governance right requires jurisdiction-specific legal review before promotion or acceptance of funds.

17. Provisional Token Design (Non-Binding)

The project has discussed a possible native token named VEIL, but no token has been issued and no sale has taken place. The token is not required to validate the core privacy research and is not part of the initial open-source grant request.

17.1 Possible future utility hypotheses

- Fee denomination or fee-payment abstraction for L2 users, subject to protocol support and a safe conversion design.
- Staking or bonding for selected network roles, if the role's performance and misconduct can be objectively measured.
- Compensation for prover work, based on verified proof jobs and transparent reward rules.
- Governance participation over narrowly defined protocol parameters, subject to capture and participation risks.

17.2 Illustrative supply discussion

A 1,000,000,000-token maximum supply and a 5% public-sale allocation have been discussed as provisional planning assumptions in earlier concept work. These figures are not approved tokenomics, not a valuation, not a sale offer, and not a commitment. They require economic simulation, legal review, and community scrutiny. They are intentionally excluded from the initial research milestones.

17.3 No price or return promises

VEIL will not promise appreciation, buyback support, guaranteed liquidity, fixed returns, or guaranteed exchange listings. Any future fee-sharing, buyback, or burn policy must be based on lawful, realized net revenue and disclosed risks. Token design should not compromise the security or neutrality of the protocol.

18. Grant-Suitable Open-Source Work Packages

For an ecosystem grant or technical Office Hours discussion, VEIL should propose a bounded work package with public deliverables. The following work packages can stand independently of any token or commercial launch.

Work package	Scope	Public deliverables	Acceptance evidence
WP1: Private note model	Specify note, commitment, nullifier, and encrypted payload lifecycle.	Protocol spec; threat model; test vectors.	Peer review; deterministic test suite.

Work package	Scope	Public deliverables	Acceptance evidence
WP2: Private transfer prototype	Implement a minimal circuit and wallet-side test harness.	Open-source prototype; sample transactions; developer guide.	Valid/invalid proof tests; no-double-spend tests.
WP3: Public/private accounting	Model conversion between public balances and private notes.	State machine; accounting invariants; simulator.	Invariant tests; reconciliation report.
WP4: Bridge message model	Specify deposit, claim, withdrawal, replay protection, and escrow invariants.	Bridge state machine; test contracts or simulator.	Replay and double-release tests; reserve accounting tests.
WP5: Privacy evaluation	Document metadata leakage and wallet/indexer privacy boundaries.	Privacy threat model; user guidance; evaluation methodology.	Independent review and documented residual risks.

Recommended initial grant scope: WP1 plus a narrowly defined portion of WP2. This creates a useful, auditable research output without promising a complete L2 before the architecture is validated.

Grant funds, if any, should be used only for the approved open-source scope and in accordance with the grant agreement. The project should not represent grant funding as investment capital or use it for an undisclosed token sale.

Appendix A. Glossary

Term	Meaning
L1 / L2	Layer 1 settlement network / Layer 2 execution or scaling system.
Rollup	A system that executes transactions outside L1 and posts data and/or proofs to L1 under defined security assumptions.
ZK proof	A cryptographic proof that a statement is true without revealing all underlying private inputs.
Note	A private value object controlled by a secret and represented on-chain by a commitment.
Commitment	A cryptographic digest binding note data while hiding its contents.
Nullifier	A value published when a note is spent to prevent double spending without directly exposing the note.
Witness	Private and public inputs used by a prover to construct a proof.
Sequencer	A component that orders transactions and proposes L2 blocks or batches.
Prover	A process or operator that generates cryptographic proofs.
Data availability (DA)	The ability for users and nodes to obtain the data required to reconstruct or verify system state.
Viewing key	A key that permits viewing some private transaction information without granting spend authority, depending on its design.
TGE	Token Generation Event; a future token-related milestone, not scheduled or committed in this draft.

Appendix B. Reference Starting Points

The following are starting points for technical research. They are not endorsements, compatibility confirmations, or claims that VEIL has integrated these projects. Verify the current documentation, release versions, and licenses before implementation.

- Ethereum developer documentation — scaling — https://ethereum.org/en/developers/docs/scaling/
- Ethereum developer documentation — zero-knowledge rollups — https://ethereum.org/en/developers/docs/scaling/zk-rollups/

• Aztec documentation — https://docs.aztec.network/

• Noir documentation — https://noir-lang.org/docs

• OpenZeppelin Contracts documentation — https://docs.openzeppelin.com/contracts/5.x/

• Safe documentation — https://docs.safe.global/

• Ethereum Improvement Proposals — https://eips.ethereum.org/

Disclaimer

This document is a preliminary technical research proposal prepared for discussion. It is not legal, financial, tax, investment, or security advice; it is not an offer to sell or a solicitation to buy any token; and it does not create any contractual commitment. All architecture, timelines, token assumptions, and implementation choices are provisional. No network deployment, token issuance, public sale, audit, partnership, regulatory approval, or performance result is claimed. Any future deployment or token-related activity requires independent technical review and jurisdiction-specific legal advice.